

CS 246 Watopoly — DD1 Plan

Cristophe Chen, Brian Fu, Kevin Wang

This document pairs with `uml.pdf` and explains the classes, their responsibilities, and how the project will be implemented. It also answers the design questions posed in the Watopoly project specification.

1. Goals & Scope

- 2–6 player, turn-based Monopoly variant ("Watopoly") with a 40-square board themed around the University of Waterloo campus.
- Text-based board display showing square names, player tokens, and improvement markers.
- Full command interpreter supporting rolling, trading, improving, mortgaging, bankruptcy, auctions, saving, and loading.
- Testing mode (`-testing`) allowing deterministic dice rolls via `roll <die1> <die2>` .
- Game state persistence: save to file and load from file (`-load <file>`).
- Robust input handling — misspelled or invalid commands should not crash the program.
- Design for extensibility: easy to add new square types, new commands, new display modes, and rule changes with minimal recompilation.

2. High-Level Architecture

See `uml.pdf` for the full class diagram.

Subsystem	Key Classes	Purpose
Core Engine	<code>WatopolyGame</code> , <code>Board</code> , <code>Player</code> , <code>Dice</code>	Central game loop, board state, player state, dice rolling and doubles tracking.
Squares	<code>Square</code> hierarchy (<code>AcademicBuilding</code> , <code>Residence</code> , <code>Gym</code> , <code>CollectOSAP</code> , <code>DCTimsLine</code> , <code>GoToTims</code> , <code>GooseNesting</code> , <code>TuitionSquare</code> , <code>CoopFee</code> , <code>SLCSquare</code> , <code>NeedlesHallSquare</code>)	Landing logic for each of the 40 board squares.
Ownership & Economy	<code>Property</code> (abstract), <code>MonopolyBlock</code> , <code>Auction</code> , <code>Trade</code>	Property ownership, monopoly grouping, improvement management, auction bidding, and trade negotiation.
Command Layer	<code>CommandInterpreter</code> , <code>Command</code> hierarchy	Parsing user input, dispatching to the appropriate game action, and handling invalid input gracefully.
Persistence	<code>SaveManager</code>	Serialization and deserialization of the full game state to/from the specified file format.
Presentation	<code>BoardDisplay</code> , <code>DisplayObserver</code>	Text rendering of the board, with an Observer interface so additional display modes can be added without modifying game logic.
Randomness	<code>RandomEventGenerator</code>	Centralized RNG for dice, SLC, Needles Hall, and Roll Up the Rim cups. Supports seeding for reproducibility.

`WatopolyGame` is the central controller. It owns the `Board`, the list of `Player` s, the `Dice`, the `CommandInterpreter`, the `RandomEventGenerator`, and the `BoardDisplay`. After each command, the display is redrawn.

3. Class Responsibilities

3.1 Core Engine

- **`WatopolyGame`** — The top-level controller. Parses command-line arguments (`-load`, `-testing`, optional `-seed`). Manages the main game loop: prompts the current player for input, delegates to the `CommandInterpreter`, enforces turn order, checks win conditions (last player standing), and triggers display redraws. Tracks the current player, whether the current player can still roll (doubles logic), and whether a player is in a "must pay" state (owing money, deciding bankruptcy vs. raising funds). Also manages the global Roll Up the Rim cup count (max 4 active).
- **`Board`** — Owns an array of 40 `Square` pointers. Provides `getSquare(int position)` to retrieve a square by index, and maps square names to positions for save/load. Responsible for initializing all 40 squares in the correct order with their correct properties (costs, tuition tables, monopoly assignments). The `Board` does not perform game logic — it is a container for the square objects and a Subject for display observers.
- **`Player`** — Stores: name, display character, cash (starts at \$1500), board position (0–39), list of owned `Property` pointers, Roll Up the Rim cup count, and

DC Tims Line state (in line or not, number of turns spent in line). Provides methods for: `addCash` / `deductCash`, `moveTo(position)` / `moveBy(steps)`, `addProperty` / `removeProperty`, `declareBankruptcy`, `getTotalWorth` (cash + printed property values + improvement costs for Tuition calculation), `isInTimsLine`, `enterTimsLine`, `leaveTimsLine`. Does not own game logic — just stores and mutates player state when told to by the game controller.

- **Dice** — Wraps the RNG to produce two die rolls (1–6 each in normal mode, or user-specified values in testing mode). Tracks the number of consecutive doubles rolled by the current player (resets on turn change). Provides `roll()` returning a `DiceResult` struct with `die1`, `die2`, `sum`, and `isDoubles`. In testing mode, `roll(int d1, int d2)` allows deterministic rolls where dice values are not restricted to 1–6.

3.2 Squares

- **Square** (abstract) — Base class for all 40 board positions. Provides a `name()` accessor and a pure virtual `landOn(Player&, WatopolyGame&)` method that defines what happens when a player lands on the square. Non-property squares implement `landOn` directly; property squares delegate to ownership/tuition/fee logic.
- **Property** (abstract, extends `Square`) — Base class for ownable squares (academic buildings, residences, gyms). Stores: `owner` (pointer to `Player`, or null if unowned), `purchaseCost`, and `isMortgaged` flag. Provides: `getOwner`, `setOwner`, `mortgage` (gives owner half of purchase cost, sets mortgaged flag), `unmortgage` (costs 60% of purchase cost), and a pure virtual `calculateFee(Player& visitor, WatopolyGame& game)` method. The `landOn` implementation in `Property` handles the common flow: if unowned, offer to buy or auction; if owned by another player and not mortgaged, charge the fee; if owned by the landing player or mortgaged, do nothing.
- **AcademicBuilding** (extends `Property`) — Adds: `monopolyBlock` (pointer to `MonopolyBlock`), `improvementCost`, `numImprovements` (0–5), and a tuition table (array of 6 values for 0–5 improvements). `calculateFee` returns the tuition based on `numImprovements`; if the owner holds the entire monopoly and `numImprovements == 0`, the base tuition is doubled. Provides `buyImprovement` (up to 5, requires owning the full monopoly, costs `improvementCost`) and `sellImprovement` (refunds half of `improvementCost`). Improvements are represented by `I` characters on the board display.
- **Residence** (extends `Property`) — Purchase cost is always \$200. `calculateFee` checks how many residences the owner holds (1→\$25, 2→\$50, 3→\$100, 4→\$200). No improvements.
- **Gym** (extends `Property`) — Purchase cost is always \$150. `calculateFee` rolls two dice (via the game's `Dice` object) and multiplies the sum by 4 (if owner has 1 gym) or 10 (if owner has 2 gyms). No improvements.
- **CollectOSAP** — `landOn` gives the player \$200 (the passing bonus is handled separately in the movement logic of `Player::moveBy`, which checks if the player crosses position 0).
- **DCTimsLine** — `landOn` does nothing if the player simply lands on it. The "sent to Tims" logic (from Go to Tims, SLC, or triple doubles) is handled by the game controller calling `Player::enterTimsLine`. The game controller also handles the "leaving Tims" procedure at the start of a jailed player's turn: attempt to roll doubles, pay \$50, or use a Roll Up the Rim cup. On the third turn in line, the player must leave.
- **GoToTims** — `landOn` moves the player to position 10 (DC Tims Line) and calls `Player::enterTimsLine`. The player does not collect \$200 even if they pass Collect OSAP.
- **GooseNesting** — `landOn` prints a flavour message ("You have been attacked by a flock of nesting geese!") and does nothing else.
- **TuitionSquare** — `landOn` prompts the player to choose between paying \$300 or 10% of their total worth (cash + printed property values + improvement costs). Payment goes to the Bank. While making this decision, `assets` and `all` commands are disabled.
- **CoopFee** — `landOn` immediately deducts \$150 from the player. Payment goes to the Bank.
- **SLCSquare** — `landOn` generates a random event from the SLC probability distribution (see Table 2 in the spec). There is a 1% chance (if fewer than 4 cups are active) of receiving a Roll Up the Rim cup instead of the normal effect. If the player is moved, play proceeds as if they landed on the destination square (recursive `landOn` call on the new square). Prints a message indicating the movement.
- **NeedlesHallSquare** — `landOn` generates a random money change from the Needles Hall probability distribution (see Table 3 in the spec). There is a 1% chance (if fewer than 4 cups are active) of receiving a Roll Up the Rim cup instead. Prints a message indicating the change.

3.3 Ownership & Economy

- **MonopolyBlock** — Groups a set of `AcademicBuilding` pointers that form a monopoly (e.g., `Arts1 = {AL, ML}`). Provides `isOwnedBy(Player&)` (checks if a single player owns all buildings in the block) and `hasAnyImprovements()` (checks if any building in the block has improvements, relevant for trade restrictions). There are 10 monopoly blocks total.
- **Auction** — Manages a single auction for one property. Takes a list of eligible players (all non-bankrupt players). Each player in turn order may raise the current bid or withdraw. When only one player remains, they pay their final bid and receive the property. Handles edge cases: all players withdraw (property remains unowned), starting bid of \$0, etc.
- **Trade** — Encapsulates a trade offer between two players. Validates that: the offer is not money-for-money, the offering player owns the offered property (if applicable), the receiving player owns the requested property (if applicable), and no property in a monopoly with improvements is being traded. If valid, presents the offer to the target player for `accept` or `reject`. On acceptance, transfers the assets between both players.

3.4 Command Layer

- **CommandInterpreter** — Reads input from stdin, parses the command and arguments, and dispatches to the appropriate `Command` object. Handles unknown/misspelled commands gracefully with an error message (no crash). Maintains a registry of valid commands. In testing mode, enables the `roll <die1> <die2>` variant.
- **Command** (abstract) — Base class with `execute(WatopolyGame&, std::vector<std::string> args)`. Concrete subclasses:

Command Class	Trigger	Action
<code>RollCommand</code>	<code>roll</code> / <code>roll d1 d2</code>	Rolls dice, moves player, triggers square's <code>landOn</code> . Handles doubles (roll again) and triple doubles (go to Tims).

Command Class	Trigger	Action
NextCommand	next	Ends the current player's turn, advances to the next player. Only valid when the player cannot roll.
TradeCommand	trade <name> <give> <receive>	Creates and processes a Trade between the current player and the named player.
ImproveCommand	improve <property> buy/sell	Buys or sells an improvement on the specified academic building. Validates monopoly ownership and improvement limits.
MortgageCommand	mortgage <property>	Mortgages the specified property. Validates ownership and that no improvements exist on the property (or its monopoly block).
UnmortgageCommand	unmortgage <property>	Unmortgages the specified property. Charges 60% of purchase cost.
BankruptCommand	bankrupt	Declares bankruptcy. Only available when the player owes more than they have. Transfers assets to creditor or auctions them if owed to the Bank.
AssetsCommand	assets	Displays the current player's assets (properties, cash, improvements). Disabled during Tuition decision.
AllCommand	all	Displays all players' assets. Disabled during Tuition decision.
SaveCommand	save <filename>	Writes the full game state to the specified file in the required format.

3.5 Persistence

- **SaveManager** — Handles reading and writing game state files. `save(watopolyGame&, std::string filename)` serializes: number of players, each player's data (name, char, tims cups, money, position, DC Tims Line state), and each ownable property's data (owner, improvements/mortgage status) in board order. `load(std::string filename)` reconstructs the full game state from a file, creating players and assigning property ownership/improvements accordingly. Validates file format and rejects invalid states (e.g., player on square 30, improvement count out of range, too many Tims cups).

3.6 Presentation

- **DisplayObserver** (abstract) — Interface with a `notify()` or `update()` method. Allows the game to trigger redraws without knowing the concrete display type.
- **BoardDisplay** (extends `DisplayObserver`) — Renders the text-based board to stdout. Reads the current state of all 40 squares and all players to produce the grid layout shown in the spec (with player characters on their current square and improvement `I` markers on academic buildings). The display logic is self-contained; it queries the `Board` and `Player` objects for current state and formats the output. The board is redrawn after every command.

3.7 Randomness

- **RandomEventGenerator** — Wraps `std::default_random_engine`. Seeded once at startup (from command-line argument or system clock). Provides methods: `rollDie()` (1–6), `generateSLCEvent()` (returns movement offset or special action per Table 2 probabilities), `generateNeedlesHallEvent()` (returns money change per Table 3 probabilities), `rollUpTheRimCheck()` (returns true with 1% probability, subject to the 4-cup global cap). Centralizing all RNG ensures reproducibility when a seed is provided.

4. Board Layout & Square Ordering

The 40 squares are arranged clockwise starting from Collect OSAP:

Position	Square Name	Type
0	Collect OSAP	Non-property
1	AL	Academic Building (Arts1)
2	SLC	Non-property
3	ML	Academic Building (Arts1)
4	Tuition	Non-property
5	MKV	Residence
6	ECH	Academic Building (Arts2)
7	Needles Hall	Non-property

Position	Square Name	Type
8	PAS	Academic Building (Arts2)
9	HH	Academic Building (Arts2)
10	DC Tims Line	Non-property
11	RCH	Academic Building (Eng)
12	PAC	Gym
13	DWE	Academic Building (Eng)
14	CPH	Academic Building (Eng)
15	UWP	Residence
16	LHI	Academic Building (Health)
17	SLC	Non-property
18	BMH	Academic Building (Health)
19	OPT	Academic Building (Health)
20	Goose Nesting	Non-property
21	EV1	Academic Building (Env)
22	Needles Hall	Non-property
23	EV2	Academic Building (Env)
24	EV3	Academic Building (Env)
25	V1	Residence
26	PHYS	Academic Building (Sci1)
27	B1	Academic Building (Sci1)
28	CIF	Gym
29	B2	Academic Building (Sci1)
30	Go to Tims	Non-property
31	EIT	Academic Building (Sci2)
32	ESC	Academic Building (Sci2)
33	Needles Hall	Non-property
34	C2	Academic Building (Sci2)
35	REV	Residence
36	Needles Hall	Non-property
37	MC	Academic Building (Math)
38	Coop Fee	Non-property
39	DC	Academic Building (Math)

5. Game Flow Overview

5.1 Startup

1. `WatopolyGame` parses command-line arguments: `-load <file>`, `-testing`, and optionally `-seed <n>`.
2. If `-load` is specified, `SaveManager::load` reconstructs the game state from file. Otherwise, the game prompts for the number of players (2–6), each player's name, and their chosen display character.
3. The `Board` is initialized with all 40 squares. `MonopolyBlock` objects are created and linked to their corresponding `AcademicBuildings`.
4. The `RandomEventGenerator` is seeded (from `-seed` argument or system clock).
5. The `BoardDisplay` is registered as an observer on the `Board` and an initial board is drawn.

5.2 Turn Loop

1. The game announces whose turn it is.
2. **Pre-roll phase:** The player may issue non-roll commands (`trade`, `improve`, `mortgage`, `unmortgage`, `assets`, `all`, `save`) at any time when not in the middle of another action.
3. **Roll phase:**
 - If the player is **in the DC Tims Line**, they follow the leaving procedure:
 - Option A: Try to roll doubles. If successful, move the rolled sum and leave the line.
 - Option B: Pay \$50 to leave, then roll and move normally.
 - Option C: Use a Roll Up the Rim cup to leave, then roll and move normally.
 - On the third turn in line without rolling doubles, the player must pay \$50 or use a cup. They then move the sum from their last roll.
 - If the player is **not in the DC Tims Line**, they roll two dice. Their token moves the sum. `landOn` is called on the destination square.
 - If they rolled doubles, they roll again (returning to step 3).
 - If they roll three consecutive doubles, they are sent to DC Tims Line instead of moving on the third roll.
 - Movement past position 0 (Collect OSAP) awards \$200, unless the player was sent to DC Tims Line.
4. **Post-roll phase:** The player may continue issuing non-roll commands. When done, they enter `next` to pass control to the next player.
5. The board is redrawn after every command.

5.3 Owing Money

When a player must pay more than they currently have:

1. The player is given the opportunity to raise funds by selling improvements (`improve <prop> sell`), mortgaging properties (`mortgage <prop>`), or trading.
2. If the player raises enough money, the debt is paid and play continues.
3. If the player cannot or chooses not to raise enough money, they may declare `bankrupt`.
 - **Bankruptcy to another player:** All of the bankrupt player's assets (cash, properties, Roll Up the Rim cups) are transferred to the creditor. Mortgaged properties transferred this way immediately incur a 10% fee to the Bank, and the creditor may choose to unmortgage them now (paying the principal) or later (paying an additional 10% on top).
 - **Bankruptcy to the Bank:** All properties are returned to the market as unmortgaged and auctioned off individually. All Roll Up the Rim cups are destroyed.
4. The bankrupt player is removed from the game.

5.4 Auctions

1. Triggered when a player declines to buy a property they landed on, or when a player goes bankrupt to the Bank.
2. All remaining (non-bankrupt) players participate.
3. In turn order, each player may raise the bid or withdraw.
4. The last remaining bidder wins and pays their final bid for the property.

5.5 End Condition

The game ends when only one player remains (all others have declared bankruptcy). That player is declared the winner.

6. Design Patterns & Rationale

Observer Pattern — Board Display

The `Board` acts as the Subject. `BoardDisplay` (and any future display types) registers as an Observer. After any state change (player movement, property purchase, improvement, etc.), the `Board` notifies its observers to redraw. This decouples game logic from presentation entirely.

Strategy Pattern — Square Landing Behaviour

Each `Square` subclass implements its own `landOn` method, encapsulating the behaviour specific to that square type. The game controller simply calls `square->landOn(player, game)` without needing to know the concrete type. Adding a new square type means creating a new subclass — no modification to the game controller or other squares.

Template Method Pattern — Property Purchase Flow

The `Property` base class implements the common `landOn` flow (check if owned, offer to buy, charge fee, etc.) as a template method. Subclasses override

`calculateFee` to provide type-specific fee computation (tuition tables for academic buildings, residence count for residences, dice-based for gyms). This avoids duplicating the purchase/ownership logic across three property types.

Factory Pattern — Board Initialization

A factory function or initializer constructs all 40 `Square` objects with their correct parameters (costs, tuition tables, monopoly assignments) and places them in the `Board` array. This centralizes the data-heavy initialization and makes it straightforward to modify board layout or add new squares.

7. Answers to Project Questions

Q1: After reading the Buildings subsection, would the Observer Pattern be a good pattern to use when implementing a game board? Why or why not?

Answer: Yes, the Observer Pattern is a strong fit for the game board. The board's visual representation needs to update whenever the game state changes — players move, properties change ownership, improvements are built or sold, players go bankrupt, etc. Rather than having every piece of game logic explicitly call a "redraw" function (which would create tight coupling between game logic and display code), we make the `Board` a Subject that notifies registered `DisplayObserver`s after each state change.

This provides several concrete benefits:

1. **Decoupling:** Game logic classes (`WatopolyGame`, `Player`, `Square` subclasses) do not need to know about the display at all. They modify state, and the Observer mechanism handles the rest.
2. **Extensibility:** If we later wanted to add a graphical display, a logging observer, or a network observer (for remote play), we simply create a new `DisplayObserver` subclass and register it — no changes to the game logic.
3. **Single responsibility:** The `Board` class focuses on maintaining square data and player positions. The `BoardDisplay` class focuses solely on rendering. Neither has to understand the other's internals.

The main consideration is performance: redrawing a text board after every single command could produce a lot of output. However, since this is a turn-based game with human-speed input, this is not a practical concern.

Q2: Suppose that we wanted to model SLC and Needles Hall more closely to Chance and Community Chest cards. Is there a suitable design pattern you could use? How would you use it?

Answer: The **Strategy Pattern** (or equivalently, the **Command Pattern**) would be well-suited for this.

Instead of hardcoding the probability distributions and effects directly inside `SLCSquare::landOn` and `NeedlesHallSquare::landOn`, we would model each possible outcome as its own class implementing a common `Card` interface:

```
class Card {
public:
    virtual void execute(Player& player, WatopolyGame& game) = 0;
    virtual std::string description() const = 0;
    virtual ~Card() = default;
};
```

Concrete cards would include:

- `MoveForwardCard(int n)` — moves the player forward by `n` squares.
- `MoveBackwardCard(int n)` — moves the player backward by `n` squares.
- `GoToTimsCard` — sends the player to DC Tims Line.
- `AdvanceToOSAPCard` — moves the player to Collect OSAP (collecting \$200).
- `GainMoneyCard(int amount)` — adds money to the player.
- `LoseMoneyCard(int amount)` — deducts money from the player.
- `RollUpTheRimCard` — gives the player a Roll Up the Rim cup.

A `CardDeck` class would hold a shuffled collection of `Card` objects (with quantities matching the specified probability distributions). `SLCSquare` and `NeedlesHallSquare` would each own a `CardDeck` and, on `landOn`, draw the top card and call `card->execute(player, game)`.

This approach has several advantages:

- **Adding new effects** (e.g., "Go directly to Goose Nesting", "Collect \$500") requires only a new `Card` subclass and adding it to the deck — no modification to `SLCSquare` or `NeedlesHallSquare`.
- **Changing probabilities** is a matter of adjusting how many of each card type are in the deck.
- **Deck behaviour** (e.g., draw-without-replacement and reshuffle when empty, like real Monopoly) can be encapsulated in `CardDeck`.
- The pattern mirrors the physical card decks in actual Monopoly, making the code intuitive.

Q3: Is the Decorator Pattern a good pattern to use when implementing Improvements? Why or why not?

Answer: The Decorator Pattern is **not** a good fit for implementing improvements in Watopoly. While it might seem appealing at first glance (each improvement "wraps" the building and modifies its tuition), there are several reasons why a simpler design is preferable:

1. **Improvements are uniform and countable.** Each academic building has exactly 6 tuition tiers (0–5 improvements), and the tuition for each tier is a fixed value from a lookup table. The Decorator Pattern is most useful when decorators add diverse, composable behaviours (e.g., adding scrollbars, borders, and shadows to a UI widget). Here, each "decorator" would do the same thing — change the tuition to the next value in the table — making the wrapping overhead

- pointless.
2. **Selling improvements requires unwrapping.** Improvements can be sold in reverse order. With the Decorator Pattern, selling the most recent improvement means removing the outermost wrapper, which is awkward in a linked chain of decorators. With a simple integer counter, selling is just `numImprovements--`.
 3. **Querying the improvement count must be easy.** The board display, save/load, trade validation, and mortgage validation all need to quickly check how many improvements a building has. With decorators, this requires traversing the chain and counting layers. With an integer field, it is `0(1)`.
 4. **No combinatorial variety.** In Watopoly, the 5 improvements are sequential (bathroom 1, bathroom 2, bathroom 3, bathroom 4, cafeteria). There is no scenario where a building could have a cafeteria but only 2 bathrooms, or where improvements from different "types" are mixed. The Decorator Pattern shines when decorations can be composed in arbitrary combinations; that flexibility is not needed here.
- Our approach:** Each `AcademicBuilding` stores an `int numImprovements` (0–5) and a tuition lookup table (array of 6 values). `calculateFee` simply indexes into the table. `buyImprovement` increments the counter (after validating monopoly ownership and the max of 5). `sellImprovement` decrements it and refunds half the improvement cost. This is simple, efficient, and easy to understand.

8. Key Design Decisions & Resilience to Change

8.1 Adding New Square Types

All squares inherit from the `Square` abstract class and implement `landOn`. To add a new square type (e.g., a "Scholarship" square that gives a random reward), one creates a new subclass, implements `landOn`, and adds it to the board initialization. No existing classes need modification.

8.2 Adding New Commands

Each command is a subclass of `Command` with an `execute` method. The `CommandInterpreter` maintains a registry mapping command strings to `Command` objects. To add a new command, create the subclass and register it — no modification to the interpreter's parsing logic or other commands.

8.3 Changing Tuition/Rent/Fee Calculations

All fee calculations are encapsulated in the respective `Property` subclass's `calculateFee` method. Tuition values are stored in data tables, not hardcoded into control flow. Changing a value means updating the table; changing the formula means modifying one method in one class.

8.4 Changing the Number of Players

The game supports 2–6 players via a dynamic list of `Player` objects. Turn order iterates through this list and skips bankrupt players. No part of the code assumes a fixed number of players.

8.5 Adding Graphics Display

Because the display uses the Observer Pattern, adding a graphical display means creating a new `DisplayObserver` subclass (e.g., `GraphicsDisplay`) and registering it alongside `BoardDisplay`. The game logic is completely unaware of how many or what kind of observers exist.

8.6 Changing Board Layout

The board is initialized by a factory that creates all 40 squares and places them in an array. Rearranging squares, adding new ones, or removing existing ones is a matter of changing the initialization code. The rest of the game operates on `Square` pointers by position index and does not assume any particular ordering.

8.7 Changing SLC / Needles Hall Probabilities

The probability distributions are stored as data in the `RandomEventGenerator` (or in the square classes themselves). Changing probabilities is a data change, not a logic change. If we adopt the Strategy/Card pattern described in Q2, it becomes even simpler — just adjust the card counts in the deck.

8.8 Changing the Save/Load Format

All serialization logic is contained within `SaveManager`. If the file format changes, only `SaveManager` needs to be updated. The game's internal representation is not coupled to the file format.

9. Implementation Roadmap

Part	Dates	Cristophe Chen	Brian Fu	Kevin Wang
1	Day 1	Finalize UML draft + class relationships for <code>WatopolyGame</code> , <code>Board</code> , and <code>Player</code> .	Finalize UML draft + command layer classes (<code>CommandInterpreter</code> , <code>Command</code> hierarchy).	Finalize UML draft + square/property hierarchy (<code>Square</code> , <code>Property</code> , specialized squares).
2	Day 2	Implement project skeleton (<code>order.txt</code> , module files, shared	Implement <code>Square</code> base + non-property squares (<code>CollectOSAP</code> ,	Implement remaining non-property squares (<code>DCTimeLine</code> , <code>GoToTims</code> , <code>SLC</code> ,

Part	Dates	enums/structs including <code>DiceResult</code>). Cristophe Chen	<code>GooseNesting</code> , <code>CoopFee</code>). Brian Fu	<code>NeedlesHall</code> , <code>Tuition</code>). Kevin Wang
3	Day 3	Implement <code>Property</code> base class + <code>MonopolyBlock</code> .	Implement <code>AcademicBuilding</code> (tuition table, improvement buy/sell rules).	Implement <code>Residence</code> + <code>Gym</code> fee logic and ownership hooks.
4	Day 4	Implement <code>WatopolyGame</code> core turn state + doubles/triple-doubles logic.	Implement <code>CommandInterpreter</code> skeleton (<code>roll</code> , <code>next</code> , <code>parser</code> validation).	Implement Board initialization (40-square order) + <code>BoardDisplay</code> text rendering hookup.
5	Day 5	Implement purchase flow + <code>Auction</code> bidding loop.	Implement economy commands (<code>improve</code> , <code>mortgage</code> , <code>unmortgage</code>) with rule checks.	Integrate movement + landing effects + OSAP pass/land handling.
6	Day 6	Implement trading system (<code>trade</code> , offer validation, accept/reject flow).	Implement bankruptcy flow to player/Bank, including mortgaged transfer rules and auctions.	Implement DC Tims Line complete flow (roll/pay/cup/third-turn forced exit).
7	Day 7	Implement <code>SaveManager::save</code> and serialization format validation.	Implement <code>SaveManager::load</code> and full state reconstruction (<code>-load</code> startup path).	Implement testing mode (<code>-testing</code> , <code>roll <die1> <die2></code>) and RNG seed option.
8	Day 8	Write integration tests for core game loop (movement, doubles, turns, win condition).	Write integration tests for economy (purchase, rent, improve, mortgage, bankruptcy).	Write integration tests for special squares (SLC, Needles Hall, Tims Line, Tuition, cups).
9	Day 9	Final integration pass + edge-case fixes.	Final integration pass + command robustness (invalid input and recovery).	Final integration pass + display/save-load consistency checks.
10	Day 10	Final checks + Marmoset dry run and submission prep.	Final checks + Marmoset dry run and submission prep.	Final checks + Marmoset dry run and submission prep.

This design prioritizes simplicity, extensibility, and robustness. Each class has a single, well-defined responsibility, and the use of polymorphism and the Observer Pattern ensures that new features can be added with minimal modification to existing code.